UE24CS242B: Operating Systems

Lab Report: PES-VCS Implementation

Name: Anshdeep Singh

SRN: PES1UG24CS070

Class: 4B

Phase 1 — Object Storage

Screenshot 1A — All Phase 1 tests passing:

```
gcc -o test_objects test_objects.o object.o -lcrypto
luman@lumanizure10:~/PES1UG24CS070-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b
9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe
9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
luman@lumanizure10:~/PES1UG24CS070-pes-vcs$
```

Screenshot 1B — Sharded object directory structure:

```
All Phase 1 tests passed.
luman@lumanizure10:~/PES1UG24CS070-pes-vcs$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
```

Phase 2 — Tree Objects

Screenshot 2A — All Phase 2 tests passing:

```
luman@lumanizure10:~/PES1UG24CS070-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

UE24CS242B: Operating Systems

Screenshot 2B — Raw binary tree object (xxd):

```
luman@lumanizure10:~/PES1UG24CS070-pes-vcs$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
luman@lumanizure10:~/PES1UG24CS070-pes-vcs$ xxd .pes/objects/2a/594d39232787fba8
eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf | head -20
00000000: 626c 6f62 2031 3500 5465 7374 2069 6e74  blob 15.Test int
00000010: 6567 7269 5879 0a                        egriXy.
luman@lumanizure10:~/PES1UG24CS070-pes-vcs$
```

Phase 3 — Staging Area

Screenshot 3A — init → add → status sequence:

```
luman@lumanizure10:~/PES1UG24CS070-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
luman@lumanizure10:~/PES1UG24CS070-pes-vcs$ echo "hello" > file1.txt
luman@lumanizure10:~/PES1UG24CS070-pes-vcs$ echo "world" > file2.txt
luman@lumanizure10:~/PES1UG24CS070-pes-vcs$ ./pes add file1.txt file2.txt
luman@lumanizure10:~/PES1UG24CS070-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  README.md
  untracked:  tree.h
  untracked:  index.c
  untracked:  test_objects.c
  untracked:  object.c
  untracked:  tree.c
  untracked:  test_objects
  untracked:  test_tree.c
  untracked:  .gitignore
  untracked:  test_tree
  untracked:  pes.h
  untracked:  commit.c
  untracked:  test_sequence.sh
  untracked:  Makefile
  untracked:  pes.c
  untracked:  index.h
  untracked:  commit.h

luman@lumanizure10:~/PES1UG24CS070-pes-vcs$ cat .pes/index
```

UE24CS242B: Operating Systems

Screenshot 3B — Raw index file contents:

```
luman@lumanizure10:~/PES1UG24CS070-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776706692 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776706702 6 file2.txt
luman@lumanizure10:~/PES1UG24CS070-pes-vcs$
```

Phase 4 — Commits and History

Screenshot 4A — pes log with three commits:

```
luman@lumanizure10:~/PES1UG24CS070-pes-vcs$ ./pes log
commit 4df80060f0b3fe62854de1729db56908ac50ee4ebeff501c1e4bc3dfc12956f4
Author: Anshdeep Singh <PES1UG24CS070>
Date:   1776708524

    Add farewell

commit 1e110a954cf25717546e5296eef3a9efb71410a5530a35c930a6a2b174c2a7f8
Author: Anshdeep Singh <PES1UG24CS070>
Date:   1776708518

    Add world

commit c5694c552069dca4bf5de8277350b94d534da9124d739741a8c6092a1d0baaef
Author: Anshdeep Singh <PES1UG24CS070>
Date:   1776708507

    Initial committial commit851A
```

Screenshot 4B — Object store growth after three commits:

```
luman@lumanizure10:~/PES1UG24CS070-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/1e/110a954cf25717546e5296eef3a9efb71410a5530a35c930a6a2b174c2a7f8
.pes/objects/4d/f80060f0b3fe62854de1729db56908ac50ee4ebeff501c1e4bc3dfc12956f4
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/c5/694c552069dca4bf5de8277350b94d534da9124d739741a8c6092a1d0baaef
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
```

UE24CS242B: Operating Systems

Screenshot 4C — HEAD and branch reference chain:

```
luman@lumanizure10:~/PES1UG24CS070-pes-vcs$ cat .pes/refs/heads/main
4df80060f0b3fe62854de1729db56908ac50ee4ebeff501c1e4bc3dfc12956f4
```

```
luman@lumanizure10:~/PES1UG24CS070-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
```

Final — Full integration test passing:

```
luman@lumanizure10:~/PES1UG24CS070-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: 45828bd04490... Initial commit

Log after first commit:
commit 45828bd04490c001cfbff1159c63656f8acb93448de1d4efdb778e3718329afe
Author: Anshdeep Singh <PES1UG24CS070>
Date:   1776708555

    Initial commit


--- Second Commit ---
Committed: 64d3a3be8d19... Update file.txt

--- Third Commit ---
Committed: 3d215e818937... Add farewell

--- Full History ---
commit 3d215e81893795c4527b06d9098f883975939bfb177ad71c9f2fb12a5ee91284
Author: Anshdeep Singh <PES1UG24CS070>
Date:   1776708555
```

UE24CS242B: Operating Systems

```
Author: Anshdeep Singh <PES1UG24CS070>
Date:   1776708555

    Add farewell

commit 64d3a3be8d19ed4493a9480e3d3d8d5ba0f68baec7807dc2e25e9a2cb89e4195
Author: Anshdeep Singh <PES1UG24CS070>
Date:   1776708555

    Update file.txt

commit 45828bd04490c001cfbff1159c63656f8acb93448de1d4efdb778e3718329afe
Author: Anshdeep Singh <PES1UG24CS070>
Date:   1776708555

    Initial committial commit855A


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
3d215e81893795c4527b06d9098f883975939bfb177ad71c9f2fb12a5ee91284

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/3d/215e81893795c4527b06d9098f883975939bfb177ad71c9f2fb12a5ee91284
.pes/objects/45/828bd04490c001cfbff1159c63656f8acb93448de1d4efdb778e3718329afe
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/64/d3a3be8d19ed4493a9480e3d3d8d5ba0f68baec7807dc2e25e9a2cb89e4195
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
lumap@lumanizure10:~/PES1UG24CS070-pes-vcs$ find .pes -type f | sort
```

Phase 5: Branching and Checkout

Q5.1: Implementing pes checkout <branch> To implement checkout, the system must first update the .pes/HEAD file to point to the new branch symbolic reference (e.g., ref: refs/heads/new-branch). Then, it must synchronize the working directory with the target branch's tree. This involves:

1. Deleting files in the current working directory that are tracked in the current branch but missing in the target branch.

2. Overwriting or creating files in the working directory using the blob data associated with the hashes in the target branch's tree object. Complexity: The primary complexity is ensuring atomicity. If the process is interrupted (e.g., a

crash or power failure), the working directory could be left in a corrupted, inconsistent state where some files belong to the old branch and others to the new one.

Q5.2: Detecting a "Dirty Working Directory" A "dirty" conflict is detected through a three-way comparison between the HEAD tree, the Index, and the Working Directory:

1. Index vs. Working Directory: Compare the file's current metadata (mtime, size) in the filesystem against the entries in the .pes/index. If they differ, the file has unstaged changes.

2. Index vs. Target Tree: Compare the blob hash in the Index with the hash for that same path in the target branch's tree object. If a file has unstaged changes and the target branch wants to change that file to a different version, checkout must refuse the operation to prevent the user's uncommitted work from being overwritten.

Q5.3: Detached HEAD State A "Detached HEAD" occurs when .pes/HEAD contains a raw 64-character commit hash instead of a symbolic ref: refs/heads/... string.

- Commits: You can still make commits; the new commit will simply list the current hash as its parent.

- Risk: However, since no branch reference points to these new commits, they are not part of any "history" that the system tracks by name. If you checkout a different branch, these commits become "orphaned."

- Recovery: A user can recover these by finding the orphaned hash in the terminal output or log and manually creating a branch pointing to it (e.g., echo <hash> > .pes/refs/heads/recovered-branch).

Phase 6: Garbage Collection and Space Reclamation

Q6.1: Reachability Algorithm The standard algorithm is Mark-and-Sweep:

1. Mark Phase: Start a recursive traversal from all "known roots": the HEAD commit, all branch references in refs/heads/, and any tags. Follow every commit to its parent, every commit to its tree, and every tree to its sub-trees and blobs. Store every encountered hash in a Hash Set or Bloom Filter.

2. Sweep Phase: Iterate through every object file in .pes/objects/. If an object's hash is not in the "Reachable" set, delete the file. Estimation: For 100,000 commits and 50 branches, you would visit every commit and tree object at least once. Depending on file overlap, you might visit approximately 500,000 to 1,000,000 objects to ensure a complete mark.

UE24CS242B: Operating Systems

Q6.2: GC Race Conditions It is dangerous because commit operations are not atomic at the object store level. A commit operation writes a new blob or tree object to disk before it updates the branch reference to point to it.

- The Race: If GC runs during that tiny window after the blob is written but before the branch points to the new commit, the GC will identify the new blob as "unreachable" and delete it.
- The Result: The commit operation then finishes by creating a reference to a hash that no longer exists, resulting in a corrupted repository. Git's Solution: Git uses a prune expiration or "grace period" (typically 2 weeks). GC will only delete unreachable objects that have not been modified/created within that timeframe, ensuring that "in-flight" objects are safe.